

API Lux (Whales) — Guía de integración para AZA

Índice

1. Arquitectura general.....	3
2. Autenticación.....	3
2.1 Login.....	3
2.2 Refresco de token	4
2.3 Cabeceras en cada llamada.....	4
3. El endpoint /proc	4
3.1 Acciones que usará AZA.....	4
4. Cómo leer el dataprovider y mapearlo al procedimiento.....	5
4.1 Estructura del JSON.....	5
4.2 Anatomía de un campo.....	5
4.3 Regla de mapeo campo → parámetro.....	5
5. Reglas transversales.....	6
5.1 Crear vs. actualizar cabecera (upsert por id).....	6
5.2 Fechas	6
5.3 Datos extra: null = no tocar, "" = borrar, valor = fijar	6
5.4 Respuestas de error	6
6. EXPEDICIONES (pedidos de salida).....	6
6.1 Crear / actualizar cabecera — p_expCabeceraAza, accion = ACTUALIZAR	7
6.2 Líneas — p_expPedidoLineas.....	8
6.3 Consultar expediciones — p_expedicionesAza, accion = SELECT.....	9
7. RECEPCIONES (albaranes de entrada)	9
7.1 Crear / actualizar cabecera — p_recCabeceraAza, accion = ACTUALIZAR	9
7.2 Líneas — p_recAlbaranLineas	10
7.3 Consultar recepciones — p_recepcionesAza, accion = SELECT	11
8. Flujos completos	12
8.1 Alta de un pedido de expedición	12
8.2 Alta de un albarán de recepción	12
8.3 Consulta	12
9. Buenas prácticas para AZA.....	12

10. Referencia rápida.....	13
----------------------------	----

Expediciones (pedidos de salida) y Recepciones (albaranes de entrada)

Documento de integración. Explica cómo autenticarse contra la API Lux, cómo funciona el endpoint `ProcedureController (/proc)` y cómo **crear, actualizar y consultar** albaranes de recepción y pedidos de expedición: cabeceras, líneas y datos extra. Incluye cómo leer los campos en el `*dataproducer*` y cómo se mapean a los parámetros de los procedimientos almacenados.

1. Arquitectura general

La API Lux es una capa REST (Java / JAX-RS) delante de SQL Server. **No contiene lógica de negocio**: cada llamada recibe un JSON, lo convierte en una ejecución `EXEC p_procedimiento @param='valor', ...` y devuelve el `*resultset*` del procedimiento como JSON.

```
Cliente AZA —HTTP(JSON)—> API Lux —EXEC p_xxx @param=...—> SQL Server
                                     <—resultset como JSON array—
```

Consecuencias clave para el integrador:

1. **Todo parámetro viaja como texto**. No hay tipos: fechas, números y booleanos se envían como `*string*` y el procedimiento los interpreta.
2. ``usuario`` y ``almacen`` los inyecta la API, no se envían en el body:
 - o `usuario` → sale del `token` (el usuario autenticado).
 - o `almacen` → sale de la cabecera ``Almacen`` de la petición.
3. ``accion`` es el verbo. Un mismo procedimiento crea, actualiza o consulta según el valor de `@accion`. Es el patrón central de todo Whales.
4. El nombre del procedimiento debe empezar por ``p_`` (si no, la API responde 403).
5. **Escape de comillas automático**. La API duplica cada `'` → `''` antes de ejecutar (anti inyección). No hace falta escaparlas tú.

2. Autenticación

2.1 Login

```
POST /login
Content-Type: application/json
```

```
{ "username": "usuarioAPI", "password": "*****" }
```

Respuesta (200 OK):

```
{
  "name": "usuarioAPI",
  "token": "<JWT>",
  "refreshToken": "<token de refresco>",
  "issuedAt": "...",
  "expiresAt": "...",
  "permisos": [ ... ]
}
```

- 401 → credenciales incorrectas.

- El token es un **JWT** que caduca (`expiresAt`). Se envía en todas las llamadas siguientes:

Authorization: Bearer <JWT>.

- Todos los endpoints de datos requieren el rol **`web`** (el usuario de AZA debe tenerlo).

2.2 Refresco de token

```
POST /login/refreshToken
{ "refreshToken": "<token de refresco>" }
```

Devuelve un nuevo token. Úsalo antes de que caduque el JWT en procesos largos.

2.3 Cabeceras en cada llamada

```
Authorization: Bearer <JWT>      ← identifica al usuario (parámetro @usuario)
Almacen: <codigo de almacen>    ← selecciona el almacén (parámetro @almacen)
Content-Type: application/json
```

El mismo usuario puede operar sobre distintos almacenes cambiando la cabecera Almacen.

3. El endpoint /proc

Es el único endpoint necesario para la integración de AZA.

```
PUT /proc/{nombreProcedimiento}
Authorization: Bearer <JWT>
Almacen: <almacen>
Content-Type: application/json

{ "accion": "...", "campo1": "valor1", "campo2": "valor2" }
```

Funcionamiento:

- El body es un **diccionario plano string→string**. Cada clave se convierte en `@clave`. Ejemplo:
EXEC p_expPedidoLineas @accion='INSERT', @pedido='PED-1', @referencia='REF-100',
@cantidadPedida='24', @usuario='<user>', @almacen='<alm>'.
- La API añade usuario y almacen. **No** los pongas en el body.
- Las claves deben coincidir **exactamente** (mayúsculas/minúsculas) con los nombres de parámetro del procedimiento (`@camelCase`). Una clave mal escrita se ignora (el parámetro toma su valor por defecto).
- Los parámetros que no envíes toman su **valor por defecto** en el SP. Esto es importante para los datos extra (ver §5.3).
- **Respuesta:** un **array JSON** con las filas del SELECT que devuelve el procedimiento. Las altas/modificaciones devuelven una sola fila, p. ej. [{ "mensaje": "OK", "idPedido": "123" }] o [{ "mensaje": "ERR_TRANSPORTISTA_NOT_EXISTS", "campo": "TRANSPORTISTA", "tab": "GENERAL" }]. Los valores NULL se devuelven como cadena vacía "".

3.1 Acciones que usará AZA

Operación	accion	Dónde
Crear / actualizar cabecera	ACTUALIZAR	procedimiento de cabecera (upsert: crea si el id es 0, si no actualiza)

Operación	accion	Dónde
Crear línea	INSERT	procedimiento de líneas
Actualizar línea	UPDATE	procedimiento de líneas
Consultar (listado)	SELECT	procedimiento visor
Consultar una cabecera	SELECT_ONE	procedimiento de cabecera
Consultar datos extra	SELECT_INICIO	procedimiento de cabecera

Nota: existe una acción CHECK_IMPORT en algunos procedimientos: es **exclusiva de la importación por Excel y no se usa en esta integración**. Las acciones ACTUALIZAR, INSERT y UPDATE ya realizan internamente todas las validaciones necesarias.

4. Cómo leer el dataprovider y mapearlo al procedimiento

Cada ventana del WMS se describe con un **dataprovider**: un JSON (carpeta `Whales<Cliente>/dataprovider/`) que define la ventana, **la lista de campos** y el **procedimiento** que la sirve. Para AZA es el ***catálogo de campos***: te dice qué campos existen, cómo se llaman y cómo se comportan.

4.1 Estructura del JSON

```
{
  "nombre": "expediciones",
  "titulo": "Expediciones",
  "procedimiento": "p_expedicionesAza",    // ← el SP que sirve la ventana
  "campos": [ { ... }, { ... } ],          // ← catálogo de campos
  "acciones": [ { ... } ]                 // botones (cada uno indica su procedimiento)
}
```

4.2 Anatomía de un campo

```
{
  "nombre": "pedido",           // ← nombre del parámetro/columna → @pedido en el SP
  "titulo": "Pedido",           // etiqueta visible
  "tipo": "STRING",             // STRING | TEXT | INTEGER | FLOAT | DATE | SELECT | COLOR
  "editable": true,             // si es editable en la UI
  "actualizarBBDD": true,       // si se envía de vuelta al SP al guardar
  "filtroActivo": true,         // si el campo puede filtrarse
  "filtro": "STRING",           // tipo de filtro: STRING | DATE | SELECT_MULTIPLE ...
  "accionValores": "",          // para tipo SELECT: la accion del SP que devuelve la lista de valores
  "required": false
}
```

4.3 Regla de mapeo campo → parámetro

`campo.nombre` (dataprovider) ↔ `@campo.nombre` (procedimiento) ↔ clave del JSON en `/proc`.

El mismo nombre se usa en los tres sitios. Avisos importantes:

- **El SP es la fuente de la verdad.** El dataprovider puede exponer un subconjunto de los parámetros del SP. Para la lista completa de parámetros aceptados, mira la firma `CREATE procedure` del `.sql`. En §6 y §7 tienes las firmas ya extraídas.
- **Algunos nombres del visor no coinciden con el parámetro de escritura.** Ejemplo típico: en el

visor de expediciones la columna se llama `cliente`, pero para crear/actualizar el parámetro es `@codCliente`; en recepciones el proveedor se escribe con `@codProveedor`. Usa siempre el nombre de parámetro que aparece en §6/§7.

- Campos con "tipo": "SELECT" y `accionValores` tienen una **lista de valores válidos** que se obtiene llamando al mismo SP con `accion = <accionValores>` (p. ej. `SELECT_CARGAS`). Útil para poblar desplegables o validar antes de enviar.
- Campos con "actualizarBBDD": `false` son **solo lectura / calculados** (p. ej. `volumen`, `peso`, `numPalets`): no tiene sentido enviarlos.

5. Reglas transversales

5.1 Crear vs. actualizar cabecera (upsert por id)

La acción `ACTUALIZAR` de los procedimientos de cabecera hace **las dos cosas** según el id:

- **Crear** → envía el id a "0" (`idPedido` en expedición, `idAlbaran` en recepción).
- **Actualizar** → envía el id real de la cabecera (el id que devuelve el visor `SELECT`).

5.2 Fechas

Formato ``dd/MM/yyyy`` (estilo 103) en todos los procedimientos de cabecera y línea. Si se omite la fecha, se aplica un valor por defecto (mañana en expedición, hoy en recepción).

5.3 Datos extra: `null` = no tocar, `""` = borrar, `valor` = fijar

Los campos de datos extra (y algunos de cabecera como `expedicion`, `carga`, `matricula`) tienen por defecto ``null`` en el SP, con esta semántica:

Qué envías en /proc	Efecto
No envías la clave (queda <code>null</code>)	No se modifica el valor actual
Envías la clave con <code>""</code>	Se borra el valor
Envías la clave con un valor	Se fija ese valor

Consecuencia práctica: en una modificación, envía **solo** los campos que quieras cambiar. Si mandas un campo vacío, lo estarás borrando.

5.4 Respuestas de error

Las acciones de escritura devuelven, en caso de error, una fila con `mensaje` (código `ERR_*` o texto `Error: ...`) y, en cabeceras, además `campo` y `tab` señalando el campo problemático. Éxito → `mensaje = 'OK'` más el id creado/actualizado.

6. EXPEDICIONES (pedidos de salida)

Procedimientos:

Objeto	Procedimiento	Acciones
Cabecera	p_expCabeceraAza	ACTUALIZAR (crear/actualizar), SELECT_ONE (leer), SELECT_INICIO (datos extra), SELECT_CARGAS, SELECT_SERVICE_LEVEL, select_propietarios, select_tipos, select_transportistas
Líneas	p_expPedidoLineas	INSERT, UPDATE, SELECT
Visor/listado	p_expedicionesAza	SELECT

Modelo de datos: expPedidoCabeceras, expPedidoLineas, expPedidoExtra (datos extra).

6.1 Crear / actualizar cabecera — p_expCabeceraAza, accion = ACTUALIZAR

Parámetros principales:

Parámetro	Oblig. (crear)	Descripción
accion	sí	ACTUALIZAR
idPedido	sí	"0" para crear ; id real para actualizar
pedido	no	Nº de pedido/ASN. Si vacío al crear, se autogenera PROPIETARIO_AAMMDD_N. Único por propietario
propietario	sí	Debe existir y tener expedición activa en el almacén
codCliente	sí	Código de cliente asociado al propietario
clienteNombre	no	Nombre del cliente
transportista	sí	Debe existir y estar asignado al propietario
serviceLevel	sí al crear	Nivel de servicio; debe estar asociado al transportista
tipoPedido	no	Por defecto NORMAL
fecha	no	Fecha de transporte dd/MM/yyyy. Si vacía → mañana
prioridad	no	Numérico. Por defecto 0
observaciones	no	Observaciones de cabecera
pedidoCliente	no	Referencia de pedido del cliente
bloqueo	no	Nombre de bloqueo (tabla bloqueos)

Parámetro	Oblig. (crear)	Descripción
direccion, cp, poblacion, provincia, pais, telefono, correo, contacto, razonSocial	no	Dirección de entrega. (...F = dirección de facturación)

Datos extra (misma llamada; semántica null/""/valor de §5.3): facturarTransporte, expedicion, observacionesAlbaran, observacionesAlmacen, carga, fechaEntrega, serviceLevel.

Validaciones internas: propietario existe · cliente asociado al propietario · transportista existe y asociado al propietario · pedido no duplicado · fecha válida · fechaEntrega válida · nivel de servicio asociado al transportista. Devuelve { "mensaje": "OK", "idPedido": "<id>", "pedido": "<numeroAsn>" , ... }.

Ejemplo — crear:

```
PUT /proc/p_expCabeceraAza
{
  "accion": "ACTUALIZAR", "idPedido": "0",
  "pedido": "PED-0001", "propietario": "AZA", "codCliente": "C001",
  "transportista": "SEUR", "serviceLevel": "24H", "fecha": "15/08/2026",
  "observacionesAlbaran": "Entregar en muelle 3", "fechaEntrega": "18/08/2026"
}
// → [{ "mensaje": "OK", "idPedido": "1234", "pedido": "PED-0001", ... }]
```

Ejemplo — actualizar (solo cambia el transportista; el resto no se toca):

```
PUT /proc/p_expCabeceraAza
{ "accion": "ACTUALIZAR", "idPedido": "1234",
  "propietario": "AZA", "codCliente": "C001", "transportista": "DHL", "serviceLevel": "24H" }
```

6.2 Líneas — p_expPedidoLineas

Parámetro	Descripción
accion	INSERT / UPDATE / SELECT
pedido	Nº de pedido (ASN). El SP resuelve el id de cabecera a partir de él
idParent	Alternativa a pedido: id de cabecera directamente
id	(solo UPDATE) id de la línea a modificar
referencia	Obligatoria. Debe existir para el propietario del pedido
cantidadPedida	Obligatoria. Numérica
linea	Nº de línea
lote	Solo si la referencia lo requiere
bloqueo	Nombre de bloqueo de línea
observaciones	Observaciones de línea
referenciaCliente	Referencia del cliente
vidaUtil	Fecha dd/MM/yyyy

Crear línea (por número de pedido, sin necesidad de conocer el id):

```
PUT /proc/p_expPedidoLineas
{ "accion": "INSERT", "pedido": "PED-0001",
  "linea": "1", "referencia": "REF-100", "cantidadPedida": "24", "lote": "L2026" }
// → [{ "mensaje": "OK", "id": "..."} ]
```

Actualizar línea (por id de línea):

```
PUT /proc/p_expPedidoLineas
{ "accion": "UPDATE", "id": "55010", "cantidadPedida": "30" }
```

6.3 Consultar expediciones — p_expedicionesAza, accion = SELECT

```
PUT /proc/p_expedicionesAza
{
  "accion": "SELECT",
  "propietario": "AZA",
  "estado": "TODOS",           // o lista por comas: "CREADO,EN PICKING"
  "pedido": "PED-0001%",      // filtros de texto = LIKE (usa % como comodín)
  "fecha": "01/08/2026", "fecha_fin": "31/08/2026" // rango de fecha de transporte
}
```

Filtros (opcionales, se combinan con AND; texto con LIKE): pedido, propietario, cliente, clienteNombre, estado, tipo, fecha/fecha_fin, fechaCreacion/fechaCreacion_fin, fechaCerrado/fechaCerrado_fin, expedicion, deliveryNumber, transportista, pedidoCliente, observaciones, agrupacion, prioridad, ruta, referencia, poblacion, pais, provincia.

Columnas devueltas (principales): `id` (= id de cabecera, úsalo para detalle/líneas), pedido, propietario, cliente, clienteNombre, estado, tipo, fecha, observaciones, prioridad, expedicion, transportista, fechaCreacion, fechaCerrado, líneas (servidas/total), unidades, muelle, ruta, nivelServicio, fechaEntrega, facturarTransporte, carga, serviceLevel.

Listas de valores para filtros: accion=pedido_estado (estados), accion=pedido_tipos (tipos).

Detalle de una expedición (con el id obtenido arriba):

```
// Cabecera: PUT /proc/p_expCabeceraAza { "accion": "SELECT_ONE", "idPedido": "1234" }
// Datos extra: PUT /proc/p_expCabeceraAza { "accion": "SELECT_INICIO", "idParent": "1234" }
// Líneas: PUT /proc/p_expPedidoLineas { "accion": "SELECT", "idParent": "1234" }
```

7. RECEPCIONES (albaranes de entrada)

Procedimientos:

Objeto	Procedimiento	Acciones
Cabecera	p_recCabeceraAza	ACTUALIZAR (crear/actualizar), SELECT_ONE (leer), SELECT_INICIO (datos extra), SELECT_DESCARGAS
Líneas	p_recAlbaranLineas	INSERT, UPDATE, SELECT
Visor/listado	p_recepcionesAza	SELECT

Modelo de datos: albaranes, albaranesLinea, albaranesExtra (datos extra).

7.1 Crear / actualizar cabecera — p_recCabeceraAza, accion = ACTUALIZAR

Parámetro	Oblig. (crear)	Descripción
accion	sí	ACTUALIZAR
idAlbaran	sí	"0" para crear ; id real para actualizar
albaran	no	Nº de albarán. Si vacío al crear, se autogenera PROPIETARIO_AAMMDD_N. Único por propietario
propietario	sí	Debe existir y tener recepción activa en el almacén
codProveedor	sí	Código de proveedor del propietario (o global). Si hay un solo proveedor puede omitirse
fecha	no	Fecha prevista dd/MM/yyyy. Si vacía → hoy
tipoAlbaran	no	Por defecto NORMAL. No se admite CROSSDOCK ni DIRECTO CLIENTE
ue	no	N0 → no UE; cualquier otro valor → UE
agencia	no	Transportista; debe existir
ubicacionRecepcion	no	Ubicación de una zona de recepción (o de devolución según el tipo)
empresa	no	Debe existir en empresas
bloqueo	no	Nombre de bloqueo de cabecera
observaciones	no	Observaciones del albarán

Datos extra (misma llamada; semántica null/"/valor de \$5.3): matricula (tractora), matRemolque, dni, nombre, apellidos, telefono, observacionesPDA, descarga.

Validaciones internas: hay proveedores · propietario requerido/existe/con recepción activa · proveedor coherente con el propietario · albarán no duplicado (si aplica) · tipo no CROSSDOCK/DIRECTO CLIENTE · fecha válida · ubicación de recepción válida · empresa/bloqueo válidos. Devuelve { "mensaje": "OK", "idAlbaran": "<id>", "albaran": "<nº>", ... }.

Ejemplo — crear cabecera:

```
PUT /proc/p_recCabeceraAza
{
  "accion": "ACTUALIZAR", "idAlbaran": "0",
  "albaran": "ALB-77", "propietario": "AZA", "codProveedor": "PROV1",
  "fecha": "12/08/2026", "agencia": "SEUR", "ue": "SI",
  "matricula": "1234ABC", "dni": "12345678Z", "nombre": "Juan"
}
// → [{ "mensaje": "OK", "idAlbaran": "3012", "albaran": "ALB-77", ... }]
```

7.2 Líneas — p_recAlbaranLineas

Las líneas de recepción se cuelgan del **id de albarán** (idParent). Crea antes la cabecera (§7.1) para obtener idAlbaran y úsalo aquí.

Parámetro	Descripción
accion	INSERT / UPDATE / SELECT
idParent	id del albarán (cabecera). Obligatorio
id	(solo UPDATE) id de la línea
referencia	Obligatoria . Debe existir para el propietario
piezasAlbaran	Cantidad del albarán. Numérica
linea	Nº de línea
lote	Según la referencia
fechaCaducidad	Fecha dd/MM/yyyy
observaciones	Observaciones de línea
cliente	Código de cliente (cross-dock a cliente)
entrega	Nº de pedido de expedición (numeroAsn) al que se vincula
bloqueo	Nombre de bloqueo de línea

Crear línea:

```
PUT /proc/p_recAlbaranLineas
{ "accion": "INSERT", "idParent": "3012",
  "linea": "1", "referencia": "REF-100", "piezasAlbaran": "50", "lote": "L1",
  "fechaCaducidad": "31/12/2026" }
// → [{ "mensaje": "OK", "id": "..."} ]
```

Actualizar línea:

```
PUT /proc/p_recAlbaranLineas
{ "accion": "UPDATE", "id": "88010", "piezasAlbaran": "60" }
```

7.3 Consultar recepciones — p_recepcionesAza, accion = SELECT

```
PUT /proc/p_recepcionesAza
{
  "accion": "SELECT",
  "propietario": "AZA",
  "albaran": "ALB-77%",           // texto = LIKE, % como comodín
  "estado": "TODOS",
  "fechaPrevista": "01/08/2026", "fechaPrevista_FIN": "31/08/2026"
}
```

Filtros (opcionales): albaran, matricula, proveedor, propietario, cliente, estado, tipo, observaciones, fechaPrevista/fechaPrevista_FIN, fechaRecepcion/fechaRecepcion_FIN. La consulta devuelve, entre otras columnas, `id` = id del albarán (para detalle/líneas).

Detalle de un albarán:

```
// Cabecera:    PUT /proc/p_recCabeceraAza { "accion": "SELECT_ONE", "idAlbaran": "3012" }
// Datos extra: PUT /proc/p_recCabeceraAza { "accion": "SELECT_INICIO", "idParent": "3012" }
// Líneas:      PUT /proc/p_recAlbaranLineas { "accion": "SELECT", "idParent": "3012" }
```

8. Flujos completos

8.1 Alta de un pedido de expedición

```
1. PUT /proc/p_expCabeceraAza { accion:ACTUALIZAR, idPedido:"0", propietario, codCliente,
                                transportista, serviceLevel, ... } → idPedido / pedido
2. PUT /proc/p_expPedidoLineas { accion:INSERT, pedido:"PED-0001", referencia, cantidadPedida }
    (repetir por cada línea)
```

8.2 Alta de un albarán de recepción

```
1. PUT /proc/p_recCabeceraAza { accion:ACTUALIZAR, idAlbaran:"0", propietario, codProveedor, ...
    }
                                → idAlbaran
2. PUT /proc/p_recAlbaranLineas { accion:INSERT, idParent:"<idAlbaran>", referencia, piezasAlbaran
    }
    (repetir por cada línea)
```

8.3 Consulta

```
1. PUT /proc/p_expedicionesAza / p_recepcionesAza { accion:SELECT, ...filtros } → filas con id
2. (opcional) detalle: SELECT_ONE (cabecera) + SELECT_INICIO (extra) + SELECT (líneas) por id
```

9. Buenas prácticas para AZA

6. **Cabecera primero, líneas después.** En expedición la cabecera debe existir; en recepción las líneas necesitan el `idAlbaran` que devuelve la cabecera.
 7. **Crear = id "0"; actualizar = id real.** La acción es `ACTUALIZAR` en ambos casos.
 8. **En modificaciones, envía solo lo que cambias.** Un campo de datos extra enviado vacío ("") **borra** el valor; omitirlo lo deja intacto (§5.3).
 9. **Fechas `dd/MM/yyyy`** en todas las llamadas.
 10. **Nombres de parámetro exactos.** Ojo con `codCliente` (no `cliente`) y `codProveedor`. Ante la duda, consulta la firma del `.sql` (§6/§7).
 11. **No envíes `usuario` ni `almacen`:** los pone la API (usuario = token, almacén = cabecera `Almacen`).
 12. **Catálogos:** para valores válidos de campos `SELECT` llama al SP con su acción (`SELECT_CARGAS`, `SELECT_SERVICE_LEVEL`, `SELECT_DESCARGAS`, `select_transportistas`, `pedido_estado`, `pedido_tipos`, ...).
 13. **Comprueba `mensaje`** en la respuesta: OK = correcto; cualquier otro valor es un error (con `campo/tab` en cabeceras).
 14. **Refresca el token** con `/login/refreshToken` antes de que caduque.
-

10. Referencia rápida

Objetivo	Procedimiento	accion
Crear/actualizar cabecera de expedición	p_expCabeceraAza	ACTUALIZAR (id 0 = crear)
Leer cabecera / datos extra de expedición	p_expCabeceraAza	SELECT_ONE / SELECT_INICIO
Crear/actualizar/consultar líneas de expedición	p_expPedidoLineas	INSERT / UPDATE / SELECT
Consultar expediciones	p_expedicionesAza	SELECT
Crear/actualizar cabecera de recepción	p_recCabeceraAza	ACTUALIZAR (id 0 = crear)
Leer cabecera / datos extra de recepción	p_recCabeceraAza	SELECT_ONE / SELECT_INICIO
Crear/actualizar/consultar líneas de recepción	p_recAlbaranLineas	INSERT / UPDATE / SELECT
Consultar recepciones	p_recepcionesAza	SELECT

Fuentes: código de la API Lux (whales-server/LUX — ProcedureController, AuthController, DataAccessWrapper) y procedimientos y dataproviders de whalesAza en whales-model.